

Mera idea><

End-to-End Workflow Plan — Amazon ML Challenge 2026

This document lays out the full pipeline for matching business records across three noisy data sources, from raw TSVs to the final leaderboard submission and reproducible zip package. It is built around a FAISS-based blocking stage feeding a feature-based classifier, tuned for the $F_{0.5}$ metric.

1. Pipeline at a Glance

Stage	Input	Output	Core Tool
1. Ingestion & Normalization	3 source TSVs	Cleaned dataframes	pandas
2. Blocking (candidate gen)	Normalized text	candidate_pairs.tsv	Sentence embeddings + FAISS
3. Feature Engineering	Candidate pairs	Feature matrix	pandas, jellyfish, sklearn
4. Model Training	Features + ground truth	Trained classifier (.pkl)	sklearn / XGBoost
5. Threshold Tuning	Validation predictions	Optimal cutoff	sklearn.metrics (F0.5)
6. Inference	Test candidates + model	matching_results.tsv	Same pipeline, test mode
7. Validation	Both TSVs	PASS/FAIL report	utils/validate_submission.py
8. Packaging	Code + outputs + docs	team_name_submission.zip	zip

Text flow: **Source1/2/3 TSV** -> **Normalize** -> **Embed** -> **FAISS top-k retrieval** -> **candidate_pairs.tsv** -> **Pairwise features** -> **Classifier** -> **Threshold @ F0.5** -> **matching_results.tsv** -> **Validator** -> **Leaderboard upload / Final zip**

2. Stage-by-Stage Details

Stage 1 — Ingestion & Normalization

- Load all files with `sep='\\t'` explicitly — never rely on the default comma separator.
- Lower-case, strip whitespace, and standardize common abbreviations (Ltd/Limited, Rd/Road) in a separate normalized column — keep the raw column untouched for reference.
- Build a single matching string per record: `name + " " + address + " " + country`.
- Keep `entity_id` as the join key throughout; never drop or reorder it.

Stage 2 — Blocking (Candidate Generation)

- Encode every record's matching string with a small MIT/Apache-licensed sentence embedding model (e.g. a MiniLM-class multilingual model, given US / India / France addresses).
- Build a FAISS index (IndexFlatIP or IVF/HNSW if the dataset is large) over Source 2 + Source 3 embeddings.
- For every Source 1 entity, query top-k nearest neighbors (start with `k=15-20`) across both sources combined.
- Do not filter by exact country match at this stage — France appears only in test and country labels can be noisy; treat country as a downstream feature, not a blocking filter.
- Measure `recall@k` on a held-out validation split before moving on — this sets the ceiling for everything downstream.
- Write results directly to `candidate_pairs.tsv` in the required format.

Stage 3 — Feature Engineering

- For each (Source1, candidate) pair, compute: FAISS cosine similarity, name edit distance (Levenshtein via jellyfish), name/address token Jaccard similarity, TF-IDF cosine on name+address, country-match flag, address component overlap (numbers, pin codes via regex).
- Keep feature computation vectorized in pandas where possible — this step runs on every candidate pair, so it is a common runtime bottleneck.
- Persist the feature matrix to disk (parquet/csv) so model iteration doesn't require re-running blocking every time.

Stage 4 — Model Training

- Label pairs: positives from `train_ground_truth.tsv`, negatives = retrieved-but-unmatched candidates (hard negatives, since they were similar enough to be retrieved).
- Train a gradient boosted tree classifier (sklearn `GradientBoostingClassifier` or `XGBoost`) on the engineered features — precision-focused metrics favor tree models here over a plain ANN.
- Save the trained model and any encoders/scalers as `.pkl` files, same pattern as prior sklearn projects.

- Cross-validate on a split that mirrors the real train/test structure (hold out entire Source-1 entities, not random rows, to avoid leakage).

Stage 5 — Threshold Tuning for F0.5

- F0.5 weights precision 2x over recall — do not default to a 0.5 probability cutoff.
- Sweep thresholds on the validation split using `sklearn.metrics.fbeta_score(beta=0.5)` computed per-entity, then macro-averaged (exactly as the challenge scores it).
- Remember singletons: predicting an empty match list correctly is worth a full 1.0 for that entity — don't force a match just because a candidate exists.

Stage 6 — Inference on Test Set

- Re-run Stages 1-3 on the test sources (no ground truth available).
- Apply the trained classifier + tuned threshold to produce final matches.
- Every Source 1 test entity must appear exactly once in the output, even singletons with an empty match list.

Stage 7 — Local Validation

- Always run `utils/validate_submission.py` against both output files before uploading — it catches formatting rejections for free, without spending a submission.
- Spot-check manually: no duplicate entity IDs, no Source-1-to-Source-1 self matches, every matched ID actually exists in the test set.

Stage 8 — Packaging the Final Submission

- Assemble the zip exactly as specified: `output/` (both TSVs), `code/business_entity_resolution/src`, `README.md`, `requirements.txt`, and the filled `Documentation_template.md`.
- Make the code folder genuinely reproducible end-to-end from the raw data — the top teams' packages get audited.
- Do not commit large raw data files or trained model binaries into the code folder unless explicitly required; reference paths instead.

3. Suggested Repository Structure

```
code/business_entity_resolution/
└── src/
    ├── ingestion.py      # load + normalize TSVs
    ├── blocking.py       # embeddings + FAISS retrieval
    ├── features.py       # pairwise similarity features
    ├── train.py          # classifier training + threshold tuning
    ├── infer.py          # generate matching_results.tsv
    ├── config.py / params.yaml # k, thresholds, paths
    ├── README.md         # exact reproduction steps
    └── requirements.txt   # pinned versions
```

This mirrors a standard modular ML pipeline layout: separate ingestion, transformation, training, and inference stages, each independently runnable and configured through one central params file.

4. Suggested 72-Hour Timeline

Window	Focus
Hours 0-6	Read problem statement fully; explore data; set up repo skeleton; get a trivial end-to-end submission through the validation script.
Hours 6-20	Build Stage 1-2 (ingestion + blocking); measure recall@k; iterate on embedding choice / k.
Hours 20-40	Feature engineering + first classifier; get a real scored submission on the leaderboard.
Hours 40-55	Threshold tuning for F0.5; error analysis on false merges vs missed matches; feature iteration.
Hours 55-65	Stability pass: rerun full pipeline clean, fix edge cases (singletons, France records, missing fields).
Hours 65-72	Write the methodology document, assemble the final zip, validate, submit early — never in the last hour.

5. Do's and Don'ts

Do

- Read .tsv files with an explicit tab separator every time — a silent single-column read is the most common early bug.
- Treat blocking recall@k as the ceiling for your whole pipeline and validate it before investing in the classifier.
- Tune your decision threshold specifically for F0.5, not for accuracy or plain F1.
- Predict empty match lists confidently for genuine singletons — it's worth full credit.
- Run the official validator before every leaderboard upload.
- Submit early and often; keep a known-good submission at all times.
- Keep country as a feature, not a filter, since France is unseen in training.
- Make the final code package genuinely reproducible end-to-end.

Don't

- Don't use any external database, API, or geocoding service to look up or normalize business identities — this is grounds for immediate disqualification.
- Don't hard-code or filter on {US, India} only — the test set includes France.
- Don't let your final matches include IDs that never appeared in candidate_pairs.tsv — the validator flags this as a pipeline bug.
- Don't submit duplicate entity IDs or duplicate source1_entity_id rows.
- Don't skip the write-up until the last hour — methodology clarity is explicitly weighted in review.
- Don't use a model over 8B parameters or one without an MIT/Apache-2.0 license.
- Don't commit large raw datasets or model binaries into the code submission folder.
- Don't wait until the final minutes to make your last leaderboard upload.